

Bag Packing Configurator - Technical Design Document (POC)

Project Overview

Build a web-based interactive bag packing configurator similar to the provided Figma design.

Users can:

- Select a bag size.
- Select packing cubes / pockets.
- Automatically arrange selected pockets inside the bag.
- View realistic visual placement using actual product images.
- See which pockets fit and which do not.
- Manually rearrange pockets.
- Rotate pockets.
- Add/remove products dynamically.

This is a **2D SVG-based application**.

No 3D rendering is required.

Primary Goal

Given:

- A bag with known dimensions.
- Multiple pockets with known dimensions.

The system should:

1. Find the best arrangement.
 2. Maximize utilized space.
 3. Minimize unused space.
 4. Show realistic visual representation.
 5. Allow manual adjustments.
-

Technology Stack

Rendering

SVG

Reason:

- Easy image rendering.
 - Easy drag and drop.
 - Easy rotation.
 - Easy hit testing.
 - Responsive.
 - Simple export support.
-

State Management

Zustand

Packing Engine

```
npm install maxrects-packer
```

Algorithm:

MaxRects

Features:

- Excellent space utilization.
 - Rectangle packing.
 - Rotation support.
 - Fast enough for real-time updates.
-

Core Architecture

IMPORTANT RULE

Packing engine MUST NOT use image dimensions.

Packing engine MUST ONLY use physical dimensions.

Units System

Store Everything In Real Units

Recommended:

cm

Example:

Bag:

55cm x 40cm

Pocket:

30cm x 20cm

Never Store

Bag:

600px x 400px

Pocket:

300px x 200px

Reason:

- Images may change.
 - Different resolutions break calculations.
 - Difficult to support multiple bag sizes.
-

Data Models

Bag Template

```
interface BagTemplate {  
  id: string;  
  
  name: string;  
  
  imageUrl: string;  
  
  widthCm: number;  
  heightCm: number;  
  
  packingAreaCm: {  
    x: number;  
    y: number;  
  
    width: number;  
    height: number;  
  };  
}
```

Example:

```
{  
  id: "small",  
  
  widthCm: 55,  
  heightCm: 40,  
  
  imageUrl: "/bags/small.png",  
  
  packingAreaCm: {  
    x: 2,  
    y: 2,  
    width: 48,  
    height: 35  
  }  
}
```

Why Packing Area Exists

The bag image contains:

- Borders
- Zippers
- Handles
- Wheels
- Decorative areas

Packing engine should use only the usable interior space.

Example:

```
Bag Physical Size
55 x 40 cm

Usable Packing Area
48 x 35 cm
```

Packing should occur only inside:

```
48 x 35 cm
```

Pocket Definition

```
interface Pocket {
  id: string;

  name: string;

  imageUrl: string;

  widthCm: number;
  heightCm: number;

  canRotate: boolean;
}
```

Example:

```
{
  id: "cube-large",
  name: "Large Cube",
  imageUrl: "/products/cube-large.png",
  widthCm: 30,
  heightCm: 20,
  canRotate: true
}
```

Pocket Instances

Packing engine works with individual items.

Example:

User selects:

Large Cube x 3

Convert to:

```
[
  cube-1,
  cube-2,
  cube-3
]
```

Each instance gets its own position.

Placement Model

```
interface Placement {
  id: string;
```

```
xCm: number;  
yCm: number;  
  
widthCm: number;  
heightCm: number;  
  
rotation: 0 | 90;  
  
fitted: boolean;  
}
```

Aspect Ratio Rules

MUST BE LOCKED

Pocket images represent real products.

Aspect ratio must never change.

Example:

30cm x 20cm

Aspect Ratio

1.5 : 1

Must always remain:

1.5 : 1

Never Do

30cm x 20cm

↓

30cm x 30cm

This distorts the product.

Image Dimensions

Image pixel dimensions are irrelevant.

Example:

Image
1000 x 800 px

Physical Size
30 x 20 cm

Packing engine should use:

30 x 20 cm

Only.

Rendering System

SVG Coordinate Conversion

Packing engine returns:

cm

SVG renders:

pixels

Convert using scale.

Example:

```
const SCALE = 10;
```

Meaning:

```
1 cm = 10 px
```

Conversion

```
svgX = xCm * SCALE;  
svgY = yCm * SCALE;  
svgWidth = widthCm * SCALE;  
svgHeight = heightCm * SCALE;
```

SVG Structure

```
<svg>  
  <!-- Bag -->  
  <image />  
  <!-- Pocket Layer -->  
  <g id="pockets">  
    <image />  
    <image />  
    <image />  
  </g>
```

</svg>

Auto Arrange Mode

Button:

Pack It For Me

Behavior:

1. User selects products.
2. MaxRects runs.
3. Positions calculated.
4. SVG updates.

User cannot drag items.

Algorithm owns layout.

Manual Mode

Button:

Customize Layout

Behavior:

- User can drag pockets.
- User can rotate pockets.
- User can move fitted items.
- User can attempt fitting unfitted items.

Algorithm no longer controls positions.

User controls layout.

Rotation Support

For POC:

0°
90°

Only.

Avoid:

13°
25°
44°

This significantly increases complexity.

Drag Support

Manual mode supports:

Mouse Drag

Updates:

xCm
yCm

in state.

Collision Detection

Prevent overlap.

Rectangle intersection:

```
intersects(rectA, rectB)
```

Returns:

```
true  
false
```

Placement Validation

When dragging:

Validate:

1. Inside packing area.
2. No overlap.
3. Valid rotation.

Visual Feedback

Valid Position:

```
Green Outline
```

Invalid Position:

```
Red Outline
```

Unfitted Items

Packing engine may fail to place all items.

Example:

Selected: 8

Fit: 6

Not Fit: 2

Do NOT remove them.

Store separately.

```
fittedItems[]  
unfittedItems[]
```

UI Behavior For Unfitted Items

Show:

△ Not Fitted

inside product list.

Example:

Large Cube

Selected: 3

Fitted: 2

Not Fitted: 1

User can:

- Remove item.
 - Change bag size.
 - Enter manual mode.
 - Attempt manual placement.
-

Recommended Screen Layout

Left Side

Bag Preview

Center

Interactive SVG Layout

Right Side

Product Selection
Selected Products
Summary

Bottom

Pack It For Me

Customize Layout

Reset

State Management

Store:

Bag

Products

Pocket Instances

Placements

Fitted Items

Unfitted Items

Mode

Selection

Zoom

Future Features (Not POC)

- Reserved zipper zones
- Stitch margins
- Multiple bag compartments
- Undo / Redo
- Save layout
- Export image
- Export PDF
- Touch support
- AI layout suggestions
- Product recommendations
- Layout sharing
- Multi-bag comparison

Folder Structure

```
src/
```

```
components/
```

```
  BagCanvas
```

```
  PocketImage
```

```
  ProductList
```

```
  Toolbar
```

```
  Summary
```

```
packing/
```

```
  PackingEngine
```

```
  MaxRectsAdapter
```

```
store/
```

```
  bagStore
```

```
utils/
```

```
geometry  
collision  
scaling  
rotation
```

```
types/  
  Bag  
  Pocket  
  Placement
```

```
hooks/  
  usePacking  
  useDrag
```

POC Success Criteria

The POC is successful when:

- ✓ User selects a bag
- ✓ User selects products
- ✓ Auto arrange works
- ✓ MaxRects packing works
- ✓ Product images render correctly
- ✓ Aspect ratio remains locked
- ✓ Rotation works
- ✓ Drag works
- ✓ Collision detection works
- ✓ Unfitted products are shown
- ✓ SVG updates in real time
- ✓ Manual mode works
- ✓ Layout resembles Figma design

✓ System can support multiple bag sizes without code changes

Final Architectural Decision

1. Use SVG.
2. Use MaxRects packing.
3. Store all measurements in centimeters.
4. Ignore source image pixel dimensions.
5. Lock aspect ratio for all product images.
6. Use a dedicated packing area instead of full bag image bounds.
7. Separate Auto Arrange mode and Manual mode.
8. Keep packing engine independent from rendering layer.
9. Support fitted and unfitted items.
10. Build POC first, then add advanced constraints later.